

DECISION STATEMENTS

Decision statements help a programmer to decide whether an instruction or a block of instructions has to be executed or skipped. There are four decision statements in Java as follows:

1. if statement
2. if else statement
3. if else if ladder
4. switch statement

IF STATEMENT:

if is a keyword, used in if decision statement in Java.

SYNTAX:

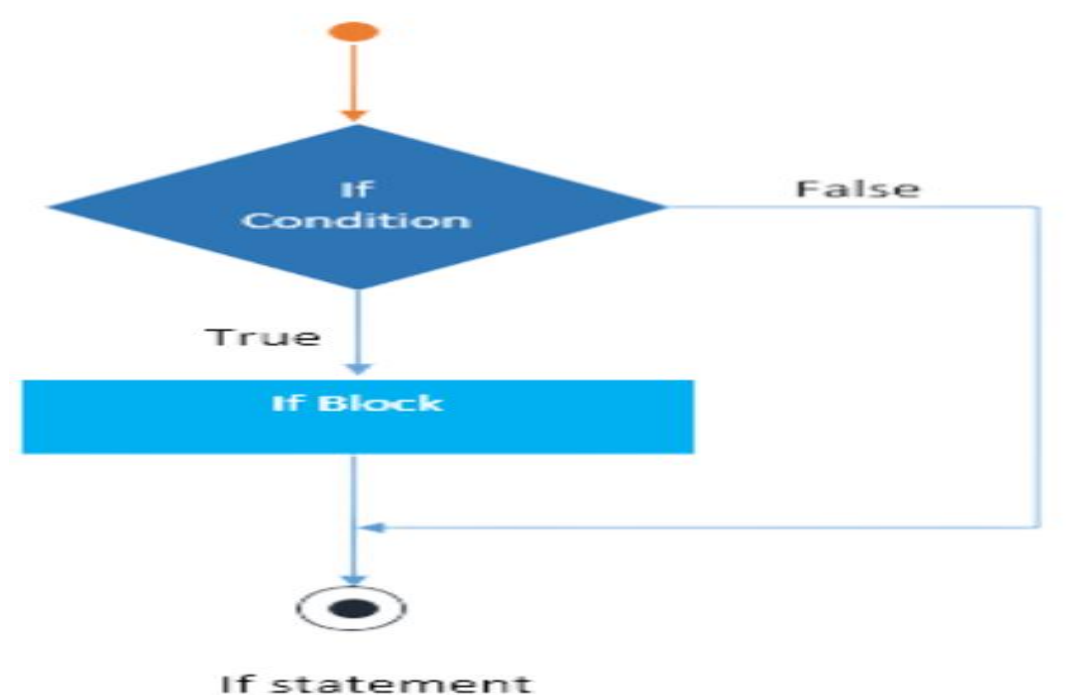
When only one statement:

```
if(condition)  
statement;
```

When more than one statement:

```
if(condition)  
{  
    //more than one statements;  
}
```

FLOW DIAGRAM:



WORKFLOW OF IF:

```
{  
    statement 1;  
    statement 2;  
    if ( condition )  
    {  
        statement 3;  
    }  
    statement 4;  
    statement 5;  
}
```

In the above workflow, statements 1 and 2 get executed.

Then if condition is checked. If condition is true, statement 3 is executed.

Otherwise, statement 3 is skipped and remaining statements 4 and 5 are executed.

```
1  class ExampleForIfStatement  
2  {  
3      public static void main(String[] args)  
4      {  
5          int age = 21;  
6          System.out.println("welcome");  
7          if(age>=18)  
8              System.out.println("you are eligible to vote");  
9          System.out.println("thank you");  
10     }  
11 }  
12
```

IF ELSE STATEMENT:

if and else are keywords, used in if else decision statement in Java.

SYNTAX:

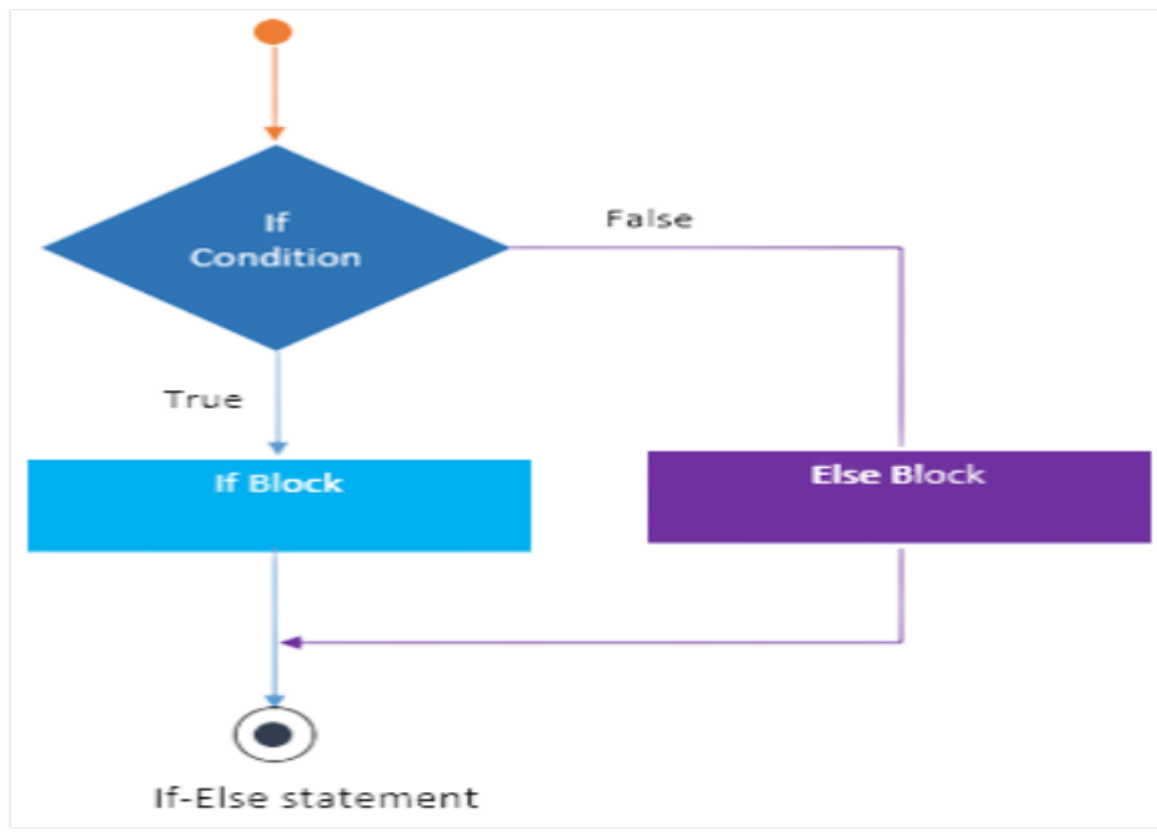
When only one statement:

```
if(condition)  
    statement;  
else  
    statement;
```

When more than one statement:

```
if(condition)  
{  
    //more than one statements;  
}  
else  
{  
    //more than one statements;  
}
```

FLOW DIAGRAM:



WORKFLOW OF IF ELSE:

```
{  
    statement 1;  
    statement 2;  
    if ( condition )  
    {  
        statement 3;  
    }  
    else  
    {  
        statement 4;  
    }  
    statement 5;  
    statement 6;  
}
```

In the above workflow, statements 1 and 2 get executed.

Then if condition is checked.

If condition is true, statement 3 is executed and statement 4 is skipped.

If condition is false, statement 3 is skipped and statement 4 is executed.

Remaining statements 4 and 5 are executed.

```
1  class ExampleForIfElseStatement
2  {
3      public static void main(String[] args)
4      {
5          int age = 14;
6          System.out.println("welcome");
7          if(age>=18)
8              System.out.println("you are eligible to vote");
9          else
10             System.out.println("you are not eligible to vote");
11             System.out.println("thank you");
12         }
13     }
14 }
```

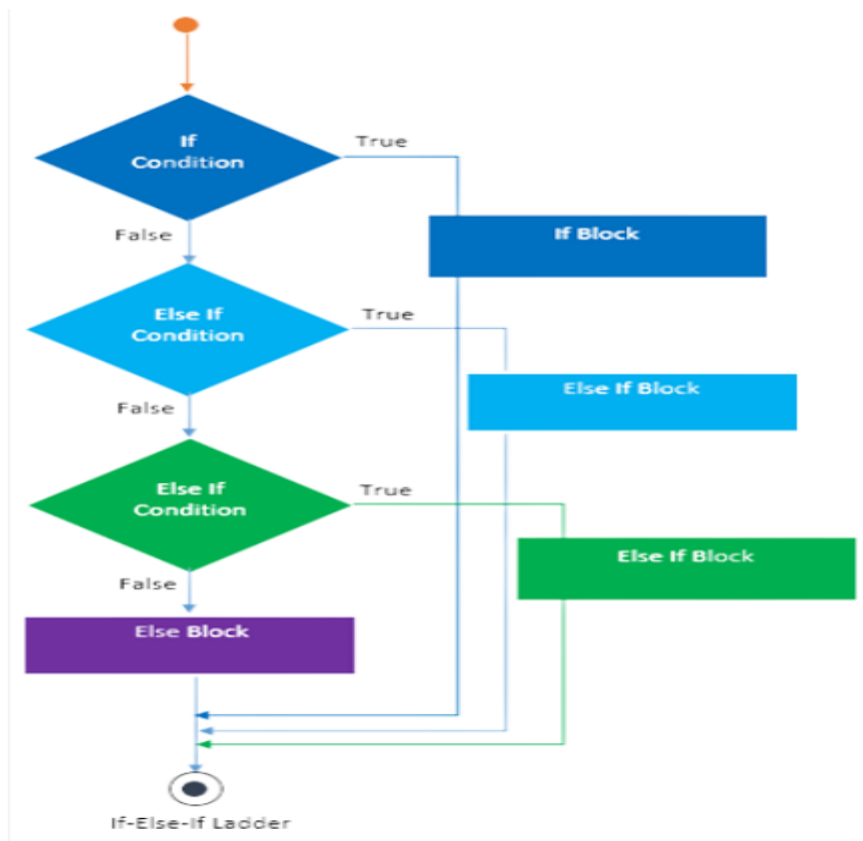
IF ELSE IF LADDER:

if and else are two keywords used in if-else-if ladder decision statement in Java. When multiple conditions have to be checked, this decision statement is used.

SYNTAX:

```
if (condition)
{
    //statements;
}
else if (condition)
{
    //statements;
}
else
{
    //statements;
}
```

FLOW DIAGRAM:



WORKFLOW OF IF ELSE IF LADDER:

```
{  
    statement 1;  
    statement 2;  
    if ( condition )  
    {  
        statement 3;  
    }  
    else if ( condition )  
    {  
        statement 4;  
    }  
    else  
    {  
        statement 5;  
    }  
}
```

```
        statement 6;  
        statement 7;  
    }
```

In the above workflow, statements 1 and 2 get executed.

Then first if condition is checked.

If condition is true, statement 3 is executed and statement 4 and 5 are skipped.

If condition is false, second if condition is checked.

If condition is true, statement 4 is executed and statement 5 is skipped.

If condition is false, else block, that is statement 5 is executed.

Remaining statements 6 and 7 are executed.

```
1  class ExampleForIfElseIfLadderStatement  
2  {  
3      public static void main(String[] args)  
4      {  
5          int marks = 68;  
6          System.out.println("welcome");  
7          if(marks>90 && marks<=100)  
8              System.out.println("S grade");  
9          else if(marks>80 && marks<=90)  
10             System.out.println("A grade");  
11         else if(marks>70 && marks<=80)  
12             System.out.println("B grade");  
13         else if(marks>60 && marks<=70)  
14             System.out.println("C grade");  
15         else if(marks>50 && marks<=60)  
16             System.out.println("D grade");  
17         else if(marks>=40 && marks<=50)  
18             System.out.println("E grade");  
19         else  
20             System.out.println("FAIL");  
21         System.out.println("thank you");  
22     }  
23 }
```

SWITCH STATEMENT:

switch is a keyword and switch statement is a special type of decision statement in Java that is different from other decision statements. The compact syntax of switch statement makes debugging easy and helps in code optimization.

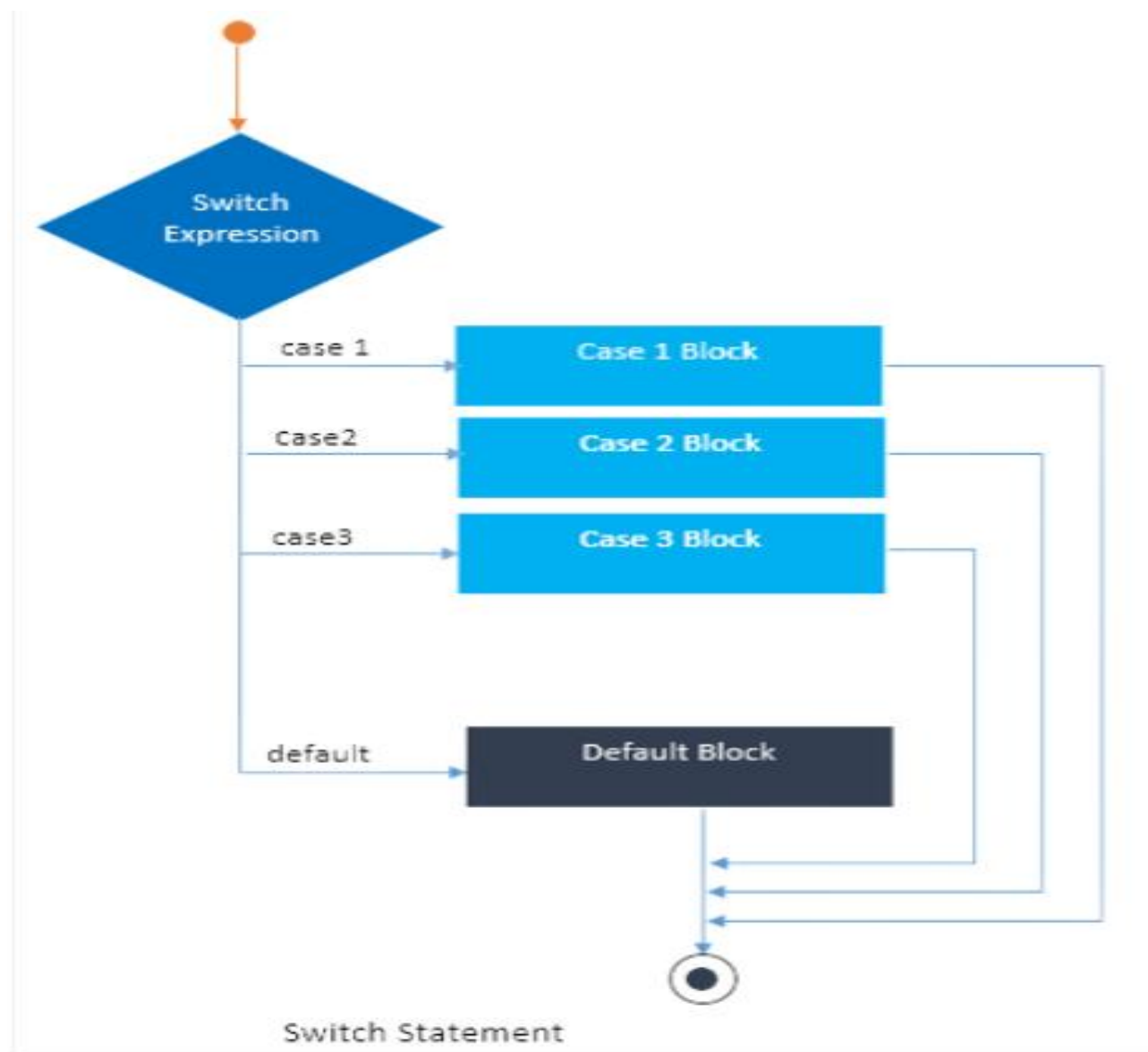
Characteristics of Switch statement:

1. The Switch statement works based on the equality principle.
2. Any number of sub-blocks called cases can be declared inside the switch block.
3. Switch datatype and Case datatype should be the same.
4. Duplicate cases are not allowed.
5. Default case is optional, but highly recommended.
6. Switch is similar to nested-if-else statement.
7. The else block in nested-if-else is similar to default case in switch.
8. We cannot use long, float, double and boolean datatypes in switch.
9. We can use byte, short, int, char, String datatypes in switch.
10. A special control transfer statement called break statement is allowed only in switch and loops in Java.
11. The advantage of switch over nested-if-else is that:
 - switch has higher performance of code due to compact syntax.
 - switch statement supports grouping of similar blocks.
12. case accepts only constant value or constant value giving expression. case cannot accept variable.

SYNTAX:

```
switch (value/expression/variable)
{
    case (value/expression) :
    {
    }
    default :
    {
    }
}
```

FLOW DIAGRAM:



WORKFLOW OF SWITCH:

```
switch(2)
{
    case 1 :
    {
        System.out.println("first option");
    }
    break;
    case 2 :
    {
        System.out.println("second option");
    }
    break;
    case 3 :
    {
        System.out.println("third option");
    }
    break;
    default :
    {
        System.out.println("invalid option");
    }
}
```

In the above example, the value 2 is passed in the switch. So, it matches with each case value and if matched, that particular block gets executed. If break statement is not used, all the following blocks get executed. The default block gets executed in all scenarios, when a case is matched or if none of the case is matched.

```

1  class SwitchExample
2  {
3      public static void main(String [] args)
4      {
5          int day = 4;
6          System.out.println("Welcome");
7          switch(day)
8          {
9              case 1:
10             {
11                 System.out.println("Sunday");
12             }
13             break;
14             case 2:
15             {
16                 System.out.println("Monday");
17             }
18             break;
19             case 3:
20             {
21                 System.out.println("Tuesday");
22             }
23             break;
24             case 4:
25             {
26                 System.out.println("Wednesday");
27             }
28             break;
29             case 5:
30             {
31                 System.out.println("Thursday");
32             }
33             break;
34             case 6:
35             {
36                 System.out.println("Friday");
37             }
38             break;
39             case 7:
40             {
41                 System.out.println("Saturday");
42             }
43             break;
44             default:
45             {
46                 System.out.println("enter a valid input");
47             }
48         }
49         System.out.println("Thank you");
50     }
51 }
52

```

These are the four types of decision statements in Java.